Flask Interview Questions & Notes

Basic Questions

1. What is Flask? Flask is a lightweight Python web framework used to build web applications and REST APIs.
2. Why is Flask called a micro framework? It provides only essential features; extra functionality comes through extensions.
3. Advantages: Lightweight, easy to learn, flexible, REST API support, large community, many extensions.
4. What is a route? A route maps a URL to a Python function.
5. What is a decorator? @app.route() associates a URL with a function.
6. What is __name__? It helps Flask locate the application and resources.
7. What does app.run() do? Starts the Flask development server.
8. Default Flask port: 5000.
9. Flask vs Django: Flask is lightweight and flexible; Django is full-stack with many built-in features.
10. What is Jinja2? Flask's template engine for dynamic HTML.

Intermediate Questions

11. Pass data to HTML using render_template().
12. render_template() loads HTML from the templates folder.
13. templates folder stores HTML files.
14. static folder stores CSS, JS, images.
15. GET retrieves data; POST sends data.
16. request.form receives form data.
17. request object contains client request data.
18. Response object is sent back to the browser.
19. redirect() sends the user to another URL.
20. url_for() generates URLs dynamically.

Advanced Questions

21. Blueprints organize large Flask apps.
22. Flask-SQLAlchemy connects Flask with SQL databases.
23. Flask-WTF helps create and validate forms.
24. Flask-Login manages authentication.
25. Sessions store user data on the server.
26. Cookies store small data in the browser.

27. REST API lets applications communicate via HTTP.
28. Common HTTP methods: GET, POST, PUT, DELETE, PATCH.
29. Create APIs using routes and jsonify().
30. jsonify() converts Python data into JSON responses.

Coding Practice

- Create a Hello World route.
- Create a dynamic route using URL parameters.
- Create a POST route.
- Return JSON using jsonify().

Interview Tips

Master Python fundamentals, Flask routes, decorators, templates, request/response, GET & POST, Jinja2, SQL/MySQL integration, REST APIs, sessions, cookies, authentication, and deployment basics.